

# Assignment #3 - Wireshark DNS & DNS Infrastructure Lab

## Wireshark DNS Lab (Part 1)

---

- **Question #1:** Run nslookup to obtain the IP address of the web server for the Indian Institute of Technology in Bombay, India: www.iitb.ac.in. What is the IP address of www.iitb.ac.in?

```
C:\Users\brooklyn>nslookup www.iitb.ac.in
Server:  one.one.one.one
Address:  1.1.1.1

Non-authoritative answer:
Name:     www.iitb.ac.in
Address:  103.21.124.133
```

- **103.21.124.133**
- **Question #2:** What is the IP address of the DNS server that provided the answer to your nslookup command in question 1 above?

```
C:\Users\brooklyn>nslookup www.iitb.ac.in
Server:  one.one.one.one
Address:  1.1.1.1

Non-authoritative answer:
Name:     www.iitb.ac.in
Address:  103.21.124.133
```

- **1.1.1.1**
- **Question #3:** Did the answer to your nslookup command in question 1 above come from an authoritative or non-authoritative server?

- **Non-authoritative**

```
C:\Users\brooklyn>nslookup www.iitb.ac.in
Server:  one.one.one.one
Address:  1.1.1.1

Non-authoritative answer:
Name:     www.iitb.ac.in
Address:  103.21.124.133
```

- **Question #4:** Use the nslookup command to determine the name of the authoritative name server for the iitb.ac.in domain. What is that name? (If there are more than one authoritative server, what is the name of the first authoritative server returned by nslookup?) If you had to find the IP address of that authoritative name server, how would you do so?
  - Name of authoritative name server for the iitb.ac.in domain: **dns1.iitb.ac.in**
  - To find the IP of this authoritative name server, I would use nslookup on dns1.iitb.ac.in
    - The IP is **103.21.125.129**

```
C:\Users\brooklyn>nslookup -type=NS www.iitb.ac.in
Server:  one.one.one.one
Address:  1.1.1.1

iitb.ac.in
    primary name server = dns1.iitb.ac.in
    responsible mail addr = postmaster.iitb.ac.in
    serial = 2013071001
    refresh = 16384 (4 hours 33 mins 4 secs)
    retry = 2048 (34 mins 8 secs)
    expire = 1048576 (12 days 3 hours 16 mins 16 secs)
    default TTL = 3960 (1 hour 6 mins)
```

```
C:\Users\brooklyn>nslookup dns1.iitb.ac.in
Server:  one.one.one.one
Address:  1.1.1.1

Non-authoritative answer:
Name:     dns1.iitb.ac.in
Address:  103.21.125.129
```

For the following 7 questions (5-11), I used the dns-wireshark-trace1-1.pcap file.

- **Question #5:** Locate the first DNS query message resolving the name "gaia.cs.umass.edu". What is the packet number in the trace for the DNS query message? Is this query message sent over UDP or TCP?

- The packet number is **#15**

|    |                 |           |             |     |                                              |
|----|-----------------|-----------|-------------|-----|----------------------------------------------|
| 15 | 15:34:40.807265 | 10.0.0.44 | 75.75.75.75 | DNS | 77 Standard query 0x3c29 A gaia.cs.umass.edu |
|----|-----------------|-----------|-------------|-----|----------------------------------------------|

- This query message is sent over **UDP**.

```

User Datagram Protocol, Src Port: 58350, Dst Port: 53
  Source Port: 58350
  Destination Port: 53
  Length: 43
  Checksum: 0xc31d [unverified]
  [Checksum Status: Unverified]
  [Stream index: 1]
  [Stream Packet Number: 1]
  > [Timestamps]
```

- **Question #6:** Now locate the corresponding DNS response to the initial DNS query. What is the packet number in the trace for the DNS response message? Is this response received via UDP or TCP?

- The packet number is **#17**

|    |                 |             |           |     |                                                            |
|----|-----------------|-------------|-----------|-----|------------------------------------------------------------|
| 17 | 15:34:40.831173 | 75.75.75.75 | 10.0.0.44 | DNS | 93 Standard query response 0x3c29 A gaia.cs.umass.edu A 12 |
|----|-----------------|-------------|-----------|-----|------------------------------------------------------------|

- This query response is sent over **UDP**.

```

  ▾ User Datagram Protocol, Src Port: 53, Dst Port: 58350
    Source Port: 53
    Destination Port: 58350
    Length: 59
    Checksum: 0x4af2 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 1]
    [Stream Packet Number: 2]
    > [Timestamps]
    UDP payload (51 bytes)

```

- **Question #7:** What is the destination port for the DNS query message? What is the source port of the DNS response message?
  - Destination Port for DNS Query: **53**

```

  > Ethernet II, Src: Apple_98:d9:27 (78:4f:43:98:d9:27), Dst: Max1
  > Internet Protocol Version 4, Src: 10.0.0.44, Dst: 75.75.75.75
  ▾ User Datagram Protocol, Src Port: 58350, Dst Port: 53
    Source Port: 58350
    Destination Port: 53

```

- Source Port for DNS Response: **53**

```

  ▾ User Datagram Protocol, Src Port: 53, Dst Port: 58350
    Source Port: 53
    Destination Port: 58350

```

- **Question #8:** To what IP address is the DNS query message sent?
  - **75.75.75.75**

```

> Frame 15: Packet, 77 bytes on wire (616 bits), 77 bytes captured
> Ethernet II, Src: Apple_98:d9:27 (78:4f:43:98:d9:27), Dst: Maxl
✓ Internet Protocol Version 4, Src: 10.0.0.44, Dst: 75.75.75.75
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 63
    Identification: 0x4c16 (19478)
> 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 255
    Protocol: UDP (17)
    Header Checksum: 0xcd5 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.44
    Destination Address: 75.75.75.75

```

- **Question #9:** Examine the DNS query message. How many “questions” does this DNS message contain? How many “answers” does it contain?
  - Questions: **1**
  - Answers: **0**

```

✓ Domain Name System (query)
    Transaction ID: 0x3c29
    > Flags: 0x0100 Standard query
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 0
✓ Queries

```

- **Question #10:** Examine the DNS response message to the initial query message. How many “questions” does this DNS message contain? How many “answers” does it contain?
  - Questions: **1**
  - Answers: **1**

```

  ▾ Domain Name System (response)
    Transaction ID: 0x3c29
    > Flags: 0x8180 Standard query response, No error
    Questions: 1
    Answer RRs: 1
    Authority RRs: 0
    Additional RRs: 0
    > Queries
    > Answers
    [Request In: 15]
    [Time: 23.908000 milliseconds]

```

- **Question #11:** The web page for the base file [http://gaia.cs.umass.edu/kurose\\_ross/](http://gaia.cs.umass.edu/kurose_ross/) references the image object [http://gaia.cs.umass.edu/kurose\\_ross/header\\_graphic\\_book\\_8E\\_2.jpg](http://gaia.cs.umass.edu/kurose_ross/header_graphic_book_8E_2.jpg), which, like the base webpage, is on [gaia.cs.umass.edu](http://gaia.cs.umass.edu). What is the packet number in the trace for the initial HTTP GET request for the base file [http://gaia.cs.umass.edu/kurose\\_ross/](http://gaia.cs.umass.edu/kurose_ross/)? What is the packet number in the trace of the DNS query made to resolve [gaia.cs.umass.edu](http://gaia.cs.umass.edu) so that this initial HTTP request can be sent to the [gaia.cs.umass.edu](http://gaia.cs.umass.edu) IP address? What is the packet number in the trace of the received DNS response? What is the packet number in the trace for the HTTP GET request for the image object [http://gaia.cs.umass.edu/kurose\\_ross/header\\_graphic\\_book\\_8E2.jpg](http://gaia.cs.umass.edu/kurose_ross/header_graphic_book_8E2.jpg)? What is the packet number in the DNS query made to resolve [gaia.cs.umass.edu](http://gaia.cs.umass.edu) so that this second HTTP request can be sent to the [gaia.cs.umass.edu](http://gaia.cs.umass.edu) IP address? Discuss how DNS caching affects the answer to this last question.

- Packet # for initial HTTP GET for Base File: Packet **#22**

|    |                 |           |                |      |                                |
|----|-----------------|-----------|----------------|------|--------------------------------|
| 22 | 15:34:40.849255 | 10.0.0.44 | 128.119.245.12 | HTTP | 831 GET /kurose_ross/ HTTP/1.1 |
|----|-----------------|-----------|----------------|------|--------------------------------|

- Packet # in the trace of the DNS query to resolve [gaia.cs.umass.edu](http://gaia.cs.umass.edu):  
Packet **#15**

|    |                 |           |             |     |                                              |
|----|-----------------|-----------|-------------|-----|----------------------------------------------|
| 15 | 15:34:40.807265 | 10.0.0.44 | 75.75.75.75 | DNS | 77 Standard query 0x3c29 A gaia.cs.umass.edu |
|----|-----------------|-----------|-------------|-----|----------------------------------------------|

- Packet # of the received DNS response: Packet **#17**

|    |                 |             |           |     |                                                            |
|----|-----------------|-------------|-----------|-----|------------------------------------------------------------|
| 17 | 15:34:40.831173 | 75.75.75.75 | 10.0.0.44 | DNS | 93 Standard query response 0x3c29 A gaia.cs.umass.edu A 12 |
|----|-----------------|-------------|-----------|-----|------------------------------------------------------------|

- Packet # for the HTTP GET request for the image object: Packet **#205**

|     |                 |           |                |      |                                                            |
|-----|-----------------|-----------|----------------|------|------------------------------------------------------------|
| 205 | 15:34:41.052343 | 10.0.0.44 | 128.119.245.12 | HTTP | 817 GET /kurose_ross/header_graphic_book_8E_2.jpg HTTP/1.1 |
|-----|-----------------|-----------|----------------|------|------------------------------------------------------------|

- Packet # in the trace of the DNS query to resolve gaia.cs.umass.edu:  
**There isn't a second one. It happens at Packet #15, the first DNS query.**
- DNS Caching Effects: **A second DNS query to resolve gaia.cs.umass.edu is not needed because the browser has an internal DNS cache that gaia.cs.umass.edu has populated.**

**For the following 4 questions (12-15), I used the dns-wireshark-trace-2-1.pcap file**

- **Question #12:** What is the destination port for the DNS query message? What is the source port of the DNS response message?
  - Destination Port: **Port 53**

```
> Frame 19: Packet, 76 bytes on wire (608 bits), 76 bytes captured (
> Ethernet II, Src: Apple_98:d9:27 (78:4f:43:98:d9:27), Dst: Maxline
> Internet Protocol Version 4, Src: 10.0.0.44, Dst: 75.75.75.75
> User Datagram Protocol, Src Port: 57837, Dst Port: 53
```

- Source Port: **Port 53**

```
> Frame 20: Packet, 92 bytes on wire (736 bits), 92 bytes captured (
> Ethernet II, Src: Maxlinear_80:00:00 (00:50:f1:80:00:00), Dst: App
> Internet Protocol Version 4, Src: 75.75.75.75, Dst: 10.0.0.44
> User Datagram Protocol, Src Port: 53, Dst Port: 57837
v Domain Name System (response)
```

- **Question #13:** To what IP address is the DNS query message sent? Is this the IP address of your default local DNS server?
  - The IP address that the DNS query message was sent to is: **75.75.75.75**
  - Yes, it is the IP address of the default local DNS server.

```

  ▾ Internet Protocol Version 4, Src: 10.0.0.44, Dst: 75.75.75.75
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 62
    Identification: 0xc2aa (49834)
    > 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0x1743 [validation disabled]
    [Header checksum status: Unverified]

```

- **Question #14:** Examine the DNS query message. What “Type” of DNS query is it? Does the query message contain any “answers”?
  - The “Type” of DNS query is a **Standard Query**, which is usually a Type A query.
  - The query contains **no** answers.

```

  ▾ Domain Name System (query)
    Transaction ID: 0x609b
    > Flags: 0x0100 Standard query
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 0
    ▾ Queries
      > www.cs.umass.edu: type A, class IN
      [Response In: 20]

```

- **Question #15:** Examine the DNS response message to the query message. How many questions does this DNS response message contain? How many answers?
  - **Questions: 1**
  - **Answers: 1**

```

  ▾ Domain Name System (response)
    Transaction ID: 0x609b
    > Flags: 0x8180 Standard query response, No error
    Questions: 1
    Answer RRs: 1

```

**For the following 3 questions (16-18), I used dns-wireshark-trace-3-1.pcap**

- **Question #16:** To what IP address is the DNS query message sent? Is this the IP address of your default local DNS server?
  - The DNS query message was sent to the IP: **75.75.75.75**
  - This is the default local DNS server
- **Question #17:** Examine the DNS query message. How many questions does the query have? Does the query message contain any "answers"?
  - **Questions: 1**
  - **Answers: 0**

```

  > Frame 13: Packet, 69 bytes on wire (552 bits), 69 bytes captured
  > Ethernet II, Src: Apple_98:d9:27 (78:4f:43:98:d9:27), Dst: Maxlin
  > Internet Protocol Version 4, Src: 10.0.0.44, Dst: 75.75.75.75
  > User Datagram Protocol, Src Port: 59963, Dst Port: 53
  ▾ Domain Name System (query)
    Transaction ID: 0x6683
    > Flags: 0x0100 Standard query
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 0
    > Queries
    [Response In: 14]

```

- **Question #18:** Examine the DNS response message (in particular the DNS response message that has type "NS"). How many answers does the response have? What information is contained in the answers? How many additional resource records are returned? What additional information is included in these additional resource records (if additional information is returned)?

- **Answers: 3**

```
Transaction ID: 0x6683
> Flags: 0x8180 Standard query response, No error
Questions: 1
Answer RRs: 3
Authority RRs: 0
Additional RRs: 3
> Queries
> Answers
\ Additional records
```

- **Information in Answers:** Names of the authoritative name servers

```
▼ Answers
> umass.edu: type NS, class IN, ns ns1.umass.edu
> umass.edu: type NS, class IN, ns ns3.umass.edu
\ Additional records
```

- **Additional Resource Record Count: 3**

```
Transaction ID: 0x6683
> Flags: 0x8180 Standard query response, No error
Questions: 1
Answer RRs: 3
Authority RRs: 0
Additional RRs: 3
> Queries
> Answers
\ Additional records
```

- **Additional Information Included In Additional Resource Records: IP Addresses of the authoritative name servers**

```
▼ Additional records
> ns2.umass.edu: type A, class IN, addr 128.119.10.28
```

---

## DNS Infrastructure SEEDS Lab (Part 2)

---

- **Task #1a**

- Zone File:

```
root@05fbaf05cf64 / # cat /etc/bind/zones/example.com.
$TTL 300
$ORIGIN example.com.
@ SOA ns1.example.com. admin.example.com. 1635647622 900 900 1800 60
ns1.example.com. A 10.154.0.71
www             A 10.154.0.72
abc             A 10.154.0.73
ns2.example.com. A 10.154.0.74
efg            A 10.154.0.75
@ NS ns1.example.com.
```

```
;; ANSWER SECTION:
abc.example.com.      300      IN      A       10.154.0.73
```

```
;; ANSWER SECTION:
www.example.com.      300      IN      A       10.154.0.72
... Query time: 0 msec
```

```
;; ANSWER SECTION:
ns2.example.com.      300      IN      A       10.154.0.74
```

```
;; ANSWER SECTION:
efg.example.com.      300      IN      A       10.154.0.75
```

- In this zone file, I have added 4 type=A records.
- ns1.example.com would be the authoritative master name server for this zone, so when we use dig, we are pinging this and asking for it to provide the IP mapped to the domain we give it.

- Observations:

- The answer section shows the IP address assigned in the zone file! We are pinging the authoritative DNS server (10.154.0.71) and asking for the domain of any of them and getting back the correctly mapped IP address.

- **Task #1b**

```

root@220faad2cc52 /etc/bind # cat named.conf.zones
zone "brooklyn2026.edu" {
    type master;
    allow-update { any; };
    file "/etc/bind/zones/brooklyn2026.edu.";
};

```

Adding "brooklyn2026.edu" zone

```

root@c245dd4518f7 /etc/bind/zones # cat brooklyn2026.edu.
$TTL 300
$ORIGIN brooklyn2026.edu.
@ SOA ns1.brooklyn2026.edu. admin.brooklyn2026.edu. 1635647669 900 900 1800 60

@ NS ns1.brooklyn2026.edu.
ns1.brooklyn2026.edu. A 10.162.0.73
cs5700                A 10.162.0.74
cy6760                A 10.162.0.75
maggie                A 10.162.0.76

```

```

;; ANSWER SECTION:
ns1.brooklyn2026.edu. 300      IN      A       10.162.0.73

```

```

;; ANSWER SECTION:
cs5700.brooklyn2026.edu. 300      IN      A       10.162.0.74

```

```

;; ANSWER SECTION:
cy6760.brooklyn2026.edu. 300      IN      A       10.162.0.75

```

```

;; ANSWER SECTION:
maggie.brooklyn2026.edu. 300      IN      A       10.162.0.76

```

- This is the same thing Task #1a, just done for scratch and for a custom named server of <NAME><YEAR>.edu
- **Task #2**
  - com. Zone File:

```

130 root@726a3077ff3e /etc/bind/zones # cat com.
$TTL 300
$ORIGIN com.
@ SOA ns1.com. admin.com. 4182639563 900 900 1800 60
ns1.com. A 10.151.0.72
@ NS ns1.com.
ns2.com. A 10.161.0.72
@ NS ns2.com.
ns1.example.com. IN A 10.154.0.71
example.com. IN NS ns1.example.com.

```

- Here, we are telling the com. zone file that the example.com. zone has the name server ns1.example.com and that ns1.example.com. is located at 10.154.0.71.

```

;; AUTHORITY SECTION:
example.com.          300      IN      NS      ns1.example.com.

;; ADDITIONAL SECTION:
ns1.example.com.     300      IN      A      10.154.0.71

```

DNS Query to Master com. server

```

;; AUTHORITY SECTION:
example.com.          300      IN      NS      ns1.example.com.

;; ADDITIONAL SECTION:
ns1.example.com.     300      IN      A      10.154.0.71

```

DNS Query to Slave com. server

- Observations:
  - We get an Authority and an Additional Section
    - The authority section tells us the namespace of the domain
    - The additional section tells us the IP of that namespace
  - No matter what the \*.example.com is, we get the same response.
    - The com. TLD server doesn't handle finding the "\*", instead it points us to the authoritative server ns1.example.com

```

root@726a3077ff3e /etc/bind/zones # dig @10.151.0.72 bleepbloop.example.com

; <<>> DiG 9.18.30-0ubuntu0.20.04.2-Ubuntu <<>> @10.151.0.72 bleepbloop.example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 34517
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 2
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 83bf1bd9f8bef7f5010000006abdce43950985db450dafbc (good)
;; QUESTION SECTION:
;bleepbloop.example.com.                IN      A

;; AUTHORITY SECTION:
example.com.                300      IN      NS      ns1.example.com.

;; ADDITIONAL SECTION:
ns1.example.com.            300      IN      A       10.154.0.71

```

- o edu. Zone File:

```

root@46737b628634 /etc/bind/zones # cat edu.
$TTL 300
$ORIGIN edu.
@ SOA ns1.edu. admin.edu. 91397176 900 900 1800 60
ns1.edu. A 10.152.0.71
@ NS ns1.edu.
brooklyn2026.edu. IN NS ns1.brooklyn2026.edu.
ns1.brooklyn2026.edu. IN A 10.162.0.73

```

- Observations: Same as above, code is also the same steps.

```

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: d9122cc6ec9e5620010000006abdf23d5b55d4d71cfa7cd1 (good)
;; QUESTION SECTION:
;gah.brooklyn2026.edu.                IN      A

;; AUTHORITY SECTION:
brooklyn2026.edu.                300      IN      NS      ns1.brooklyn2026.edu.

;; ADDITIONAL SECTION:
ns1.brooklyn2026.edu.            300      IN      A       10.162.0.73

;; Query time: 0 msec
;; SERVER: 10.152.0.71#53(10.152.0.71) (UDP)
;; WHEN: Thu Oct 01 05:40:13 UTC 2026
;; MSG SIZE rcvd: 111

```

- Task #3

```

cat /etc/named.conf
1 root@5949ac707e0c /etc/bind/zones # cat root
$TTL 300
$ORIGIN .
@ SOA ns1. admin. 567747005 900 900 1800 60
ns1. A 10.150.0.72
@ NS ns1.
ns2. A 10.160.0.72
@ NS ns2.
com. IN NS ns1.com.
ns1.com. IN A 10.151.0.72
com. IN NS ns2.com.
ns2.com. IN A 10.161.0.72
edu. IN NS ns1.edu.
ns1.edu. IN A 10.152.0.71

```

ROOT Server 1

```

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: b58a95eb77e73a94010000006abdf8f891594d03233f3ce1 (good)
;; QUESTION SECTION:
;com.                                IN      A

;; AUTHORITY SECTION:
com.                                300     IN      NS      ns2.com.
com.                                300     IN      NS      ns1.com.

;; ADDITIONAL SECTION:
ns2.com.                            300     IN      A       10.161.0.72
ns1.com.                            300     IN      A       10.151.0.72

```

Querying root IP #1 for \*.com.

```
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 009650950a898aae010000006abdfalc00f15f0c06f1830e (good)
;; QUESTION SECTION:
;test.edu.                IN      A

;; AUTHORITY SECTION:
edu.                      300     IN      NS      ns1.edu.

;; ADDITIONAL SECTION:
ns1.edu.                  300     IN      A      10.152.0.71

;; Query time: 0 msec
;; SERVER: 10.150.0.72#53(10.150.0.72) (UDP)
;; WHEN: Thu Oct 01 06:13:48 UTC 2026
;; MSG SIZE rcvd: 99
```

Querying root IP #1 for \*.edu.

```
130 root@a1847b02d718 /etc/bind/zones # cat root
$TTL 300
$ORIGIN .
@ SOA ns1. admin. 567747006 900 900 1800 60
ns1. A 10.150.0.72
@ NS ns1.
ns2. A 10.160.0.72
@ NS ns2.
com. IN NS ns1.com.
ns1.com. IN A 10.151.0.72
com. IN NS ns2.com.
ns2.com. IN A 10.161.0.72
edu. IN NS ns1.edu.
ns1.edu. IN A 10.152.0.71
```

ROOT Server 2

```
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: b52525cle965bbf8010000006abdf9e099a401236670610c (good)
;; QUESTION SECTION:
;geum.com.                IN      A

;; AUTHORITY SECTION:
com.                300      IN      NS      ns2.com.
com.                300      IN      NS      ns1.com.

;; ADDITIONAL SECTION:
ns2.com.            300      IN      A      10.161.0.72
ns1.com.            300      IN      A      10.151.0.72

;; Query time: 0 msec
;; SERVER: 10.160.0.72#53(10.160.0.72) (UDP)
;; WHEN: Thu Oct 01 06:12:48 UTC 2026
;; MSG SIZE  rcvd: 133
```

Querying root IP #2 for \*.com.

```
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: b1a3bda47e0cb08e010000006abdf9fdcdf918036d44f1e0 (good)
;; QUESTION SECTION:
;test.edu.               IN      A

;; AUTHORITY SECTION:
edu.                300      IN      NS      ns1.edu.

;; ADDITIONAL SECTION:
ns1.edu.            300      IN      A      10.152.0.71

;; Query time: 0 msec
;; SERVER: 10.160.0.72#53(10.160.0.72) (UDP)
;; WHEN: Thu Oct 01 06:13:17 UTC 2026
;; MSG SIZE  rcvd: 99
```

Querying root IP #2 for \*.edu.

#### o Observations & Code Explanation:

- I added a NS and an A record for the com. server, the com. slave server, and the edu. server.
- This tells the root server that if the user is looking for something in the "com." zone or the "edu." zone, here are the TLD servers (namespace

& IP) that you can ask for further information on the specific domain for resolution.

- I notice that we have an Authority and an Additional section again, but they only show the TLD servers, not the authoritative servers like the TLD servers did.
- We also can use an arbitrary name such as \*.edu or \*.com and we will still get an answer for the TLD server's IP and name servers.

#### • Task #4

- Initial Query to Global\_DNS #1:

```
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:;; udp: 1232
; COOKIE: 1106a808cb3eb2b6010000006abf5ae560d51d8574f4b5f4 (good)
;; QUESTION SECTION:
;example.com.                IN      A

;; AUTHORITY SECTION:
example.com.                 60      IN      SOA      ns1.example.com. adm
in.example.com. 591463082 900 900 1800 60

;; Query time: 8 msec
;; SERVER: 10.153.0.53#53(10.153.0.53) (UDP)
;; WHEN: Fri Oct 02 07:19:01 UTC 2026
;; MSG SIZE rcvd: 114
```

- Modified root.hints file on Global\_DNS #1:

```
root@84015f87d33d /usr/share/dns # cat root.hints
ns1. A 10.150.0.82
. NS ns1.
ns2. A 10.160.0.82
. NS ns2.
```

- Query to Global\_DNS #1 after Modifying root.hints File:

```
root@84015f87d33d /usr/share/dns # dig @10.153.0.53 example.com
;; communications error to 10.153.0.53#53: timed out
```

```
; <<>> DiG 9.18.30-0ubuntu0.20.04.2-Ubuntu <<>> @10.153.0.53 example
.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 15383
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: cec3cffb77f49b42010000006abf5c04c3c93bf0437da194 (good)
;; QUESTION SECTION:
;example.com.                IN      A

;; Query time: 5000 msec
;; SERVER: 10.153.0.53#53(10.153.0.53) (UDP)
;; WHEN: Fri Oct 02 07:23:48 UTC 2026
;; MSG SIZE rcvd: 68
```

- Initial Query to Global\_DNS #2:

```
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 48147
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 758f11fe01424bff010000006abf5d1e736fde4098189af2 (good)
;; QUESTION SECTION:
;example.com.                IN      A

;; AUTHORITY SECTION:
example.com.                 60      IN      SOA      ns1.example.com. adm
in.example.com. 591463082 900 900 1800 60

;; Query time: 0 msec
;; SERVER: 10.163.0.53#53(10.163.0.53) (UDP)
;; WHEN: Fri Oct 02 07:28:30 UTC 2026
;; MSG SIZE rcvd: 114
```

- Modified root.hints file on Global\_DNS #2

```
root@11cd324552b6 /usr/share/dns # cat root.hints
ns1. A 10.150.0.82
. NS ns1.
ns2. A 10.160.0.82
. NS ns2.
```

- Query to Global\_DNS #2 after Modifying root.hints File:

```
root@11cd324552b6 /usr/share/dns # dig @10.163.0.53 example.com
;; communications error to 10.163.0.53#53: timed out
```

```
; <<>> DiG 9.18.30-0ubuntu0.20.04.2-Ubuntu <<>> @10.163.0.53 example
.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 28067
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 3db2a9aa998e2b35010000006abf5cc69ce6d0731e876bca (good)
;; QUESTION SECTION:
;example.com.                IN      A

;; Query time: 4996 msec
;; SERVER: 10.163.0.53#53(10.163.0.53) (UDP)
;; WHEN: Fri Oct 02 07:27:02 UTC 2026
;; MSG SIZE rcvd: 68
```

- Observations:

- The DNS resolution process no longer works. This happens because the root level servers are no longer mapped to the correct IP address, so when the local DNS server attempts to contact the root DNS server, they don't get a (proper) response.
- This makes it so that the DNS resolution process cannot even begin as the root server can't even be contacted.

- **Task #5**

- Inside AS155 Host resolv.conf File:

```
root@06a067ddb50f / # cat /etc/resolv.conf
search neu.edu
nameserver 127.0.0.11
nameserver 10.153.0.53
nameserver 10.163.0.53
options ndots:0
```

- Dig Query for www.example.com:

```

root@06a067ddb50f / # dig www.example.com

; <<> DiG 9.16.1-Ubuntu <<> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 3817
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: ec390650c4a9be16010000006abf5e428bef6483f3030537 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                300     IN      A      10.154.0.72

;; Query time: 12 msec
;; SERVER: 10.153.0.53#53(10.153.0.53)
;; WHEN: Fri Oct 02 07:33:22 UTC 2026
;; MSG SIZE rcvd: 88

```

- Dig Query for maggie.brooklyn2026.edu:

```

root@06a067ddb50f / # dig maggie.brooklyn2026.edu

; <<> DiG 9.16.1-Ubuntu <<> maggie.brooklyn2026.edu
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 10569
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 0a0c4381d621a054010000006abf5ea94ea191fe0588f086 (good)
;; QUESTION SECTION:
;maggie.brooklyn2026.edu.        IN      A

;; ANSWER SECTION:
maggie.brooklyn2026.edu. 300     IN      A      10.162.0.76

;; Query time: 0 msec
;; SERVER: 10.153.0.53#53(10.153.0.53)
;; WHEN: Fri Oct 02 07:35:05 UTC 2026
;; MSG SIZE rcvd: 96

```

- Dig Query for doesntexist.brooklyn2026.edu:

```

; <<>> DiG 9.16.1-Ubuntu <<>> doesntexist.brooklyn2026.edu
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 48919
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 4d08077ade49e533010000006abf5ebe4ff4e4a745f1bdee (good)
;; QUESTION SECTION:
;doesntexist.brooklyn2026.edu. IN      A

;; AUTHORITY SECTION:
brooklyn2026.edu.      60      IN      SOA      ns1.brooklyn2026.edu
. admin.brooklyn2026.edu. 1635647669 900 900 1800 60

;; Query time: 0 msec
;; SERVER: 10.153.0.53#53(10.153.0.53)
;; WHEN: Fri Oct 02 07:35:26 UTC 2026
;; MSG SIZE rcvd: 131

```

- Observations:

- When querying for a specific domain under a zone that exists in an authoritative server, we get an "A" record back
- If we query for a domain that doesn't exist in a zone, we get an SOA record back from the Local DNS server.
  - It shows us this as a proof of authority.
  - We also get back an NXDOMAIN response, meaning the domain doesn't exist, here is proof it doesn't exist from the authoritative server.

- Packet Trace Description:

- The packets start at the host where we initiated the query. The host contacts the local DNS by going through the necessary network hops. The local DNS will then go through the necessary network hops to reach the root DNS, the information travels back, it traverses to the TLD DNS, the information travels back, it traverses to the authoritative DNS, the information travels back, then it goes back to the host.

- **Task #6**

- Adding in-arpa.arpa Zone to the Root DNS Server Zones File

```

1 root@3c352b57d82f / # cat /etc/bind/zones/root
$TTL 300
$ORIGIN .
@ SOA ns1. admin. 567747007 900 900 1800 60
ns1. A 10.150.0.72
@ NS ns1.
ns2. A 10.160.0.72
@ NS ns2.
com. IN NS ns1.com.
ns1.com. IN A 10.151.0.72
com. IN NS ns2.com.
ns2.com. IN A 10.161.0.72
edu. IN NS ns1.edu.
ns1.edu. IN A 10.152.0.71
in-addr.arpa IN NS ns1.com.
in-addr.arpa IN NS ns2.com.

```

- Adding in-addr.arpa Zone

```

root@2e842d36ff9f / # cat /etc/bind/named.conf.zones
zone "com." { type master; notify yes; allow-transfer { any; }; file
"/etc/bind/zones/com."; allow-update { any; }; };
zone "in-addr.arpa." {
    type master;
    notify yes;
    allow-transfer { any; };
    allow-update { any; };
    file "/etc/bind/zones/in-addr.arpa.";
};

```

- Added this in the com. nameserver

- Adding in-addr.arpa. Zone File

```
130 root@2e842d36ff9f / # cat /etc/bind/zones/in-addr.arpa.  
$TTL 300  
$ORIGIN in-addr.arpa.
```

```
@ SOA ns1.com. admin.com. 1565237346 900 900 1800 60  
@ NS ns1.com.  
@ NS ns2.com.  
ns1.com. A 10.151.0.72  
ns2.com. A 10.161.0.72
```

```
154.10.in-addr.arpa. NS ns1.example.com.  
ns1.example.com. A 10.154.0.71
```

- Adding 154.10.in-addr.arpa Zone

```
root@95826d0b3a9a /etc/bind # cat named.conf.zones  
zone "example.com." { type master; file "/etc/bind/zones/example.com  
."; allow-update { any; }; };  
zone "154.10.in-addr.arpa" {  
    type master;  
    notify yes;  
    allow-transfer { any; };  
    allow-update { any; };  
    file "/etc/bind/zones/154.10.in-addr.arpa.";  
};
```

- Adding 154.10.in-addr.arpa Zone File

```
root@95826d0b3a9a /etc/bind/zones # cat 154.10.in-addr.arpa.  
$TTL 300  
$ORIGIN 154.10.in-addr.arpa.  
@ SOA ns1.example.com. admin.example.com. 1635647622 900 900 1800 60  
  
@ NS ns1.example.com.  
ns1.example.com. A 10.154.0.71  
  
71.0 IN PTR ns1.example.com.  
72.0 IN PTR www.example.com.  
73.0 IN PTR abc.example.com.  
74.0 IN PTR ns2.example.com.  
75.0 IN PTR efg.example.com.
```

- Testing

```

1 root@ea6dccda4afe / # dig -x 10.154.0.72

; <<>> DiG 9.16.1-Ubuntu <<>> -x 10.154.0.72
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 36655
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 8fc3be48d1b3a0bb010000006abfe21efcd6acc045a8431f (good)
;; QUESTION SECTION:
;72.0.154.10.in-addr.arpa.      IN      PTR

;; ANSWER SECTION:
72.0.154.10.in-addr.arpa. 300     IN      PTR      www.example.com.

;; Query time: 48 msec
;; SERVER: 10.153.0.53#53(10.153.0.53)
;; WHEN: Fri Oct 02 16:55:58 UTC 2026
;; MSG SIZE rcvd: 110

```

```

root@ea6dccda4afe / # dig -x 10.154.0.73

; <<>> DiG 9.16.1-Ubuntu <<>> -x 10.154.0.73
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 61057
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 0d61a0d95139ad71010000006abfe22ebfa26c2b379097de (good)
;; QUESTION SECTION:
;73.0.154.10.in-addr.arpa.      IN      PTR

;; ANSWER SECTION:
73.0.154.10.in-addr.arpa. 300     IN      PTR      abc.example.com.

;; Query time: 11 msec
;; SERVER: 10.153.0.53#53(10.153.0.53)
;; WHEN: Fri Oct 02 16:56:14 UTC 2026
;; MSG SIZE rcvd: 110

```

```

root@ea6dccda4afe / # dig -x 10.154.0.74

; <<>> DiG 9.16.1-Ubuntu <<>> -x 10.154.0.74
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 49131
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: e17866303ad3833e010000006abfe23d5dcf0b56d82b8736 (good)
;; QUESTION SECTION:
;74.0.154.10.in-addr.arpa.      IN      PTR

;; ANSWER SECTION:
74.0.154.10.in-addr.arpa. 300     IN      PTR      ns2.example.com.

;; Query time: 3 msec
;; SERVER: 10.153.0.53#53(10.153.0.53)
;; WHEN: Fri Oct 02 16:56:29 UTC 2026
;; MSG SIZE rcvd: 110

```

```

root@ea6dccda4afe / # dig -x 10.154.0.75

; <<>> DiG 9.16.1-Ubuntu <<>> -x 10.154.0.75
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 36326
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: ed19d6f379967065010000006abfe24c95b4f365a41ce955 (good)
;; QUESTION SECTION:
;75.0.154.10.in-addr.arpa.      IN      PTR

;; ANSWER SECTION:
75.0.154.10.in-addr.arpa. 300     IN      PTR      efg.example.com.

;; Query time: 7 msec
;; SERVER: 10.153.0.53#53(10.153.0.53)
;; WHEN: Fri Oct 02 16:56:44 UTC 2026
;; MSG SIZE rcvd: 110

```

- Non-mapped IP

```

; <<>> DiG 9.16.1-Ubuntu <<>> -x 10.154.0.76
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 36739
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 4e93daa07ffbaa1e0100000006abfe2608f62b012c766f676 (good)
;; QUESTION SECTION:
;76.0.154.10.in-addr.arpa.      IN      PTR

;; AUTHORITY SECTION:
154.10.in-addr.arpa.      60      IN      SOA      ns1.example.com. adm
in.example.com. 1635647622 900 900 1800 60

;; Query time: 12 msec
;; SERVER: 10.153.0.53#53(10.153.0.53)
;; WHEN: Fri Oct 02 16:57:04 UTC 2026
;; MSG SIZE rcvd: 157

```

- Overall Observations:
  - The authority section acts similar to a “signature” in that it tells us this server gave us the authoritative decision on where to reach this domain / this domain doesn’t exist.
    - We can see this when we dig -x for an IP that isn’t mapped to any domain
  - Nothing extremely interesting (out of the ordinary / out of what I expected) happened, but I did learn about the authority section!
  - I didn’t know that a PTR record acts as the reverse DNS lookup (reverse of an “A” record)